PROJECT GUIDE

How the Project Works

Forecasting freeway traffic speed in Los Angeles, 5 to 60 minutes ahead, from engineered features to a graph neural network.

Dataset METR-LA: 207 road sensors, 4 months, 5-minute readings

Models Historical Average, Persistence, Ridge, Gradient Boosting, LSTM, Graph WaveNet

Best result Graph WaveNet, average error 3.07 mph over the next hour

Deliverables Python package, experiment scripts, tests, results, IEEE paper, interactive web app

This guide explains what the project does and how each part works. The detailed project report and the viva question bank are separate documents.

1. What this project is

Every five minutes, 207 loop detectors buried in the freeways of Los Angeles County report the average speed of the cars passing over them. This project asks one question: **given the last hour of readings from every sensor, what will the speed be at every sensor over the next hour?**

That is a *spatio-temporal forecasting* problem. It is *temporal* because each sensor's future depends on its own recent past and on the time of day and week. It is *spatial* because a traffic jam at one sensor spreads to its neighbours along the road. The project builds a ladder of six models, from a model that simply predicts "the usual speed for this time of week" up to **Graph WaveNet**, a graph neural network that learns how congestion moves through the road network. It then compares them fairly and tests whether the differences are statistically real.

207	34,272	12 → 12	3.07 mph
sensors (loop detectors) on LA freeways	time steps, 5 min apart (1 Mar – 27 Jun 2012)	last hour in, next hour out (12 steps of 5 min)	Graph WaveNet's average error over the next hour

Why it matters

Short-term speed forecasts drive route guidance (navigation apps), ramp metering, incident response and travel-time signs. A forecast 15 to 60 minutes ahead gives operators time to act before a jam forms.

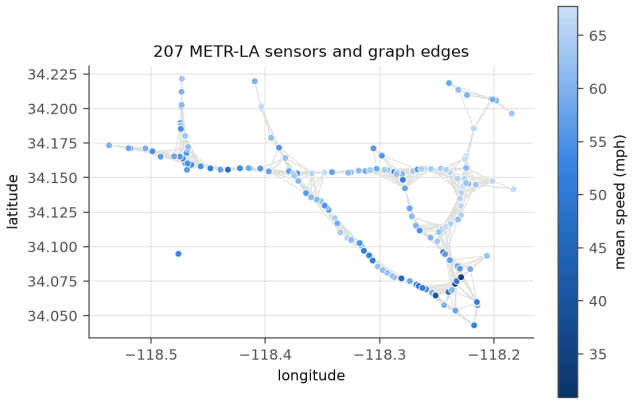
What makes it hard

- **Sudden changes.** Speed can collapse from 65 to 20 mph within minutes when congestion starts.
- **Strong daily and weekly cycles.** Weekday evenings are slow, weekends are not.
- **Congestion spreads along roads.** What happens downstream now often reaches a sensor later.
- **Missing data.** 8.1% of readings are missing because sensors fail. They are stored as 0 mph.

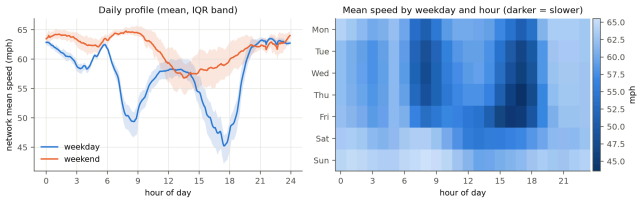
2. The data

The project uses **METR-LA**, the standard public benchmark for this task (released with the DCRNN paper, ICLR 2018). It is stored in `Dataset/` as four files:

File	What it holds
`METR-LA.h5`	The speed matrix: 34,272 time steps × 207 sensors, in mph. A value of **0 means "missing"**, not a stopped car.
`graph_sensor_ids.txt`	The order of the 207 sensor IDs (columns of the matrix).
`graph_sensor_locations.csv`	Latitude and longitude of each sensor (used for maps).
`distances_la_2012.csv`	Driving distance along the road network between pairs of sensors. This is used to build the road graph.



The sensor network. 207 detectors on several LA freeways, coloured by their mean speed (darker = slower). Grey lines are edges of the road graph.



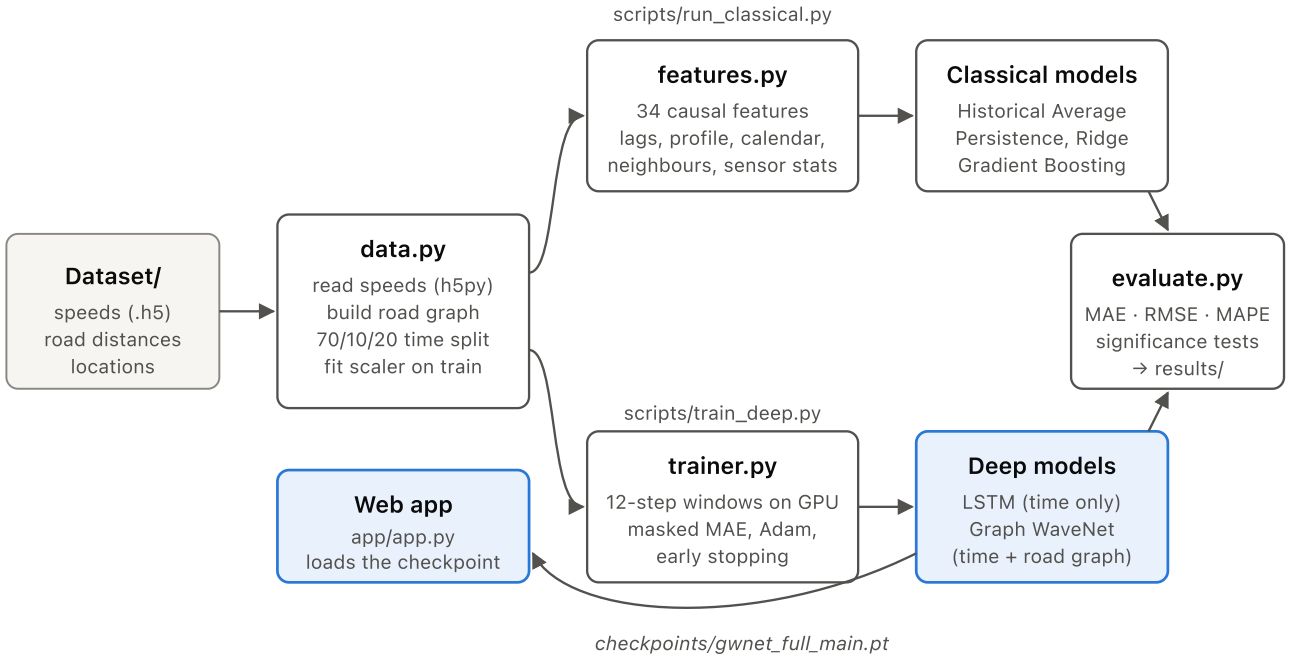
The daily rhythm. The weekday evening rush drops the network-average speed to about 45 mph around 17:30; weekends stay fast.

What the data exploration found, and what each finding changed

Finding (from `scripts/run_eda.py`)	Design decision it led to
8.1% of readings are 0 (sensor failures). Some days have network-wide outages.	Ignore zeros in every loss and score ("masking"). Fill gaps only with the *last known* value, never with future values.
Speed 5 minutes ago is highly correlated with speed now (autocorrelation 0.86).	Use recent speeds ("lags") as features.
Strong weekly cycle: the 1-week autocorrelation (0.40) is higher than the 1-day one (0.36).	Build a "usual speed for this sensor at this time of week" profile.
Nearby sensors (on the road graph) are correlated even after removing the daily cycle (0.27 vs 0.07 for unconnected pairs).	Add neighbour features and use a graph neural network.
Speeds are capped at 70 mph and skewed (most readings are fast, a tail is very slow).	Train with absolute error (MAE), which is robust to the slow tail.
The test period has more missing data (12.1%) than training (7.1%).	Report how models behave when the latest reading is missing.

3. The pipeline, end to end

The code is an installable Python package (`src/traffic/`) plus one script per experiment (`scripts/`). Data flows through it like this:



Pipeline. One loader feeds two model families. Every model is scored by the same masked metrics on the same test windows, and the trained Graph WaveNet powers the web app.

Step	Command	Output
1. Explore the data	`python scripts/run_eda.py`	EDA figures and `results/tables/eda_stats.json` (~1 min)
2. Baselines + feature models	`python scripts/run_classical.py`	Predictions and metrics for HA, Persistence, Ridge, Gradient Boosting; feature ablation and importance (~40 min, CPU)
3. Deep models	`bash scripts/run_deep_experiments.sh`	Graph WaveNet, LSTM and the Graph WaveNet ablation (~4 h on an Apple M3)
4. Compare everything	`python scripts/evaluate.py`	Result tables (CSV + LaTeX), significance tests, figures
5. Check correctness	`pytest`	23 unit tests (leakage, alignment, masking, shapes)
6. Use the model	`streamlit run app/app.py`	Interactive forecast page at `localhost:8501`

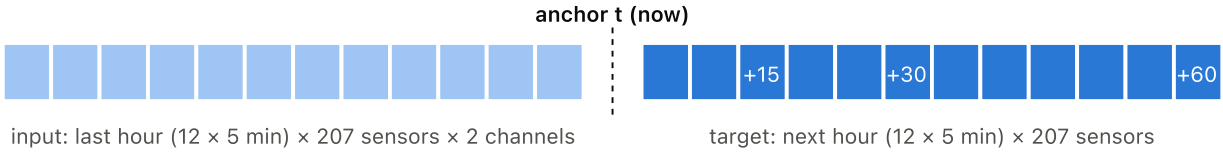
4. How each part works

4.1 Loading and cleaning (`src/traffic/data.py`)

The speed file was written by a very old pandas version (0.15) that modern pandas cannot read, so `load_speed()` reads the raw arrays with `h5py`. Zeros are kept as zeros: they are a *flag* for "missing". Later steps either skip them (losses, metrics) or fill them with the last known value (features). They are never filled from the future.

4.2 Turning a time series into training examples

Each example is a **window** anchored at a time t. The input is the 12 readings from $t - 55$ min to t; the target is the 12 readings from $t + 5$ to $t + 60$ min, for all 207 sensors at once.



One forecasting window. Results are reported at 15, 30 and 60 minutes ahead (steps 3, 6 and 12) and averaged over all 12 steps.

Windows are split **by time**, never shuffled: the first 70% train the models (1 Mar – 23 May), the next 10% are for tuning (validation, to 4 Jun) and the last 20% are the untouched test set (4 – 27 Jun). That gives 23,974 / 3,425 / 6,850 windows, exactly the split used in the published papers, so results can be compared directly. Shuffling would leak: neighbouring windows share 11 of their 12 inputs.

4.3 The road graph

Sensors are nodes; the road distance d_{ij} from sensor i to sensor j becomes an edge weight with a Gaussian kernel:

$$W_{ij} = \exp\left(-\frac{d_{ij}^2}{\sigma^2}\right) \quad \text{if } \geq 0.1, \quad \text{else } 0$$

Close sensors get weights near 1; far ones are cut to 0 to keep the graph sparse (1,515 directed edges, plus self-loops = 1,722 non-zeros, identical to the original DCRNN preprocessing). Road distance depends on direction, so the graph is *directed*: 73% of edges are one-way. Two "random-walk" matrices are derived from it: P_f passes information *downstream* and P_b passes it *upstream*.

4.4 Feature engineering (`src/traffic/features.py`)

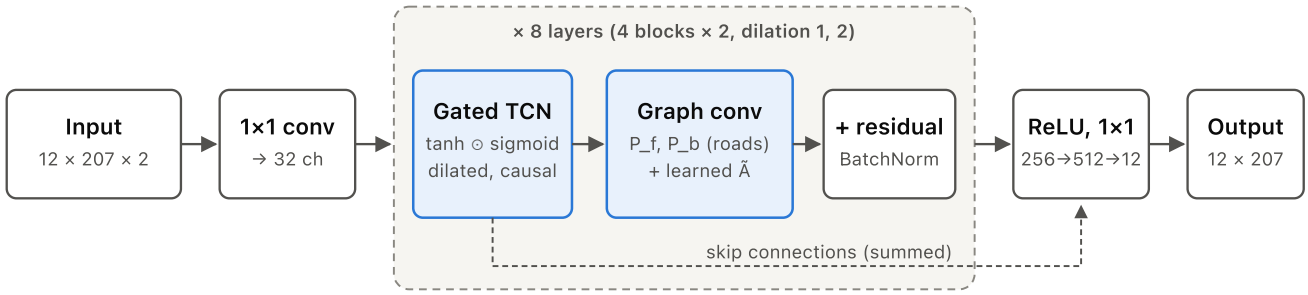
The classical models need hand-made inputs. Each row is one (time, sensor) pair, and the label is the speed h steps later. There are 34 features in six groups, and every feature uses only information available at time t (a unit test proves this by overwriting the future and checking nothing changes).

Group	#	Features	Why
Lags	12	speed at $t, t-1, \ldots, t-11$ (gaps forward-filled)	recent past predicts the near future
Window stats	7	1- and 3-step change, rolling mean/std/min, # missing, missing-now flag	trend, volatility, data quality
Calendar	4	sin/cos of time-of-day of the *target*, day of week, weekend	daily and weekly cycles
Weekly profile	3	usual speed at the target slot, usual speed now, anomaly = now − usual	split speed into "normal" + "deviation"
Spatial	5	mean speed / anomaly of downstream and upstream neighbours, downstream trend	jams travel upstream
Sensor	3	each sensor's training mean, std, missing rate	"which sensor is this"

4.5 The six models

Model	How it predicts	What it tests
Historical Average	The sensor's average training speed at that time of week.	seasonality alone
Persistence	"Speed stays as it is now" for all 12 steps.	recency alone
Ridge regression	A linear formula of the 34 features, one per horizon, with an L2 penalty for the correlated lags.	linear use of features
Gradient Boosting	Hundreds of small decision trees, each fixing the previous trees' errors (scikit-learn's histogram version, absolute-error loss), one model per horizon.	non-linear use of features
LSTM	A recurrent network that reads each sensor's last hour and outputs 12 steps. It is shared by all sensors and sees no neighbours.	learned temporal patterns, no space
Graph WaveNet	A convolutional network over time combined with graph convolution over the road network, plus a graph it learns by itself.	learned space + time

4.6 Inside Graph WaveNet (`src/traffic/models/gwnet.py`)



Graph WaveNet. Each layer first looks along time (gated, dilated convolution), then across the road network (diffusion graph convolution). Skip connections from all eight layers feed a small output head that predicts all 12 future steps at once. 300,952 parameters.

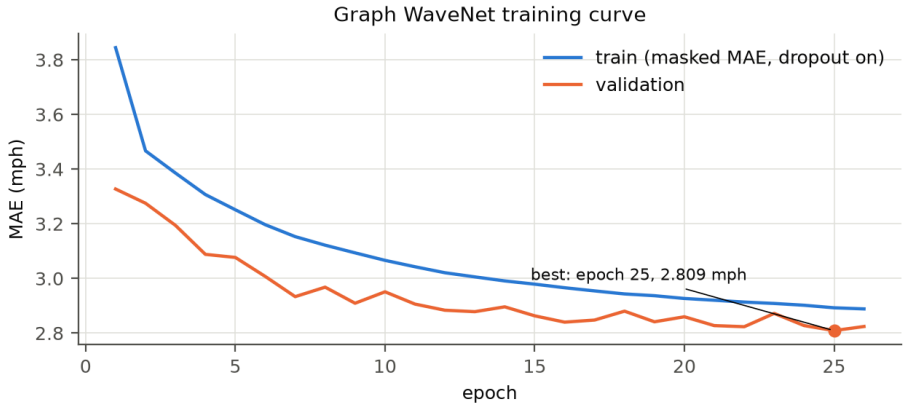
- **Gated temporal convolution:** $h = \tanh(\Theta_1 \star x) \odot \sigma(\Theta_2 \star x)$. The sigmoid "gate" decides how much signal passes, like an LSTM gate, but all time steps are computed in parallel. With dilations 1 and 2 in four blocks, every output sees the whole input hour (receptive field 13 ≥ 12).
- **Diffusion graph convolution:** each sensor mixes in information from up to 2 hops downstream (P_f) and upstream (P_b), like traffic moving along the roads.
- **Self-adaptive adjacency:** $\tilde{A} = \mathrm{softmax}(\mathrm{ReLU}(E_1 E_2))$, a third graph learned from two small embedding matrices (207 × 10). It can capture links the road graph misses.

4.7 Training (`src/traffic/trainer.py`)

Inputs have two channels per sensor and step: speed, standardised with the *training* mean and standard deviation (58.58 ± 12.83 mph), and time of day in [0, 1). The model's outputs are converted back to mph, and the loss is the **masked mean absolute error**, which skips missing targets:

$$\mathcal{L} = \frac{\sum m_{t,i}\,|\hat{y}_{t,i} - y_{t,i}|}{\sum m_{t,i}}, \qquad m_{t,i} = 1[y_{t,i} > 0]$$

Optimiser: Adam (learning rate 0.001, weight decay 0.0001), batch size 32, gradient clipping at 5. After every epoch the validation MAE is computed. The best weights are saved to `checkpoints/`, and training stops early if validation does not improve for 8 epochs. Training uses the Apple M3 GPU (MPS) when available, otherwise CUDA or the CPU.



Graph WaveNet training. Validation error fell from 3.33 to 2.81 mph; the best checkpoint is from epoch 25. The run was interrupted after epoch 26 of the planned 35, and all results use that best checkpoint. Training error sits above validation because dropout is active during training.

4.8 Evaluation (`src/traffic/metrics.py`, `scripts/evaluate.py`)

- **MAE** (average absolute error, mph), **RMSE** (penalises big misses) and **MAPE** (error in %), computed only where a real reading exists.
- Reported at 15, 30 and 60 minutes and averaged over all 12 steps, the same protocol as the literature.
- **Diebold–Mariano test:** is one model's error *significantly* lower than another's? A Newey–West correction accounts for overlapping windows.
- **Block bootstrap:** 95% confidence intervals made by resampling whole test days.

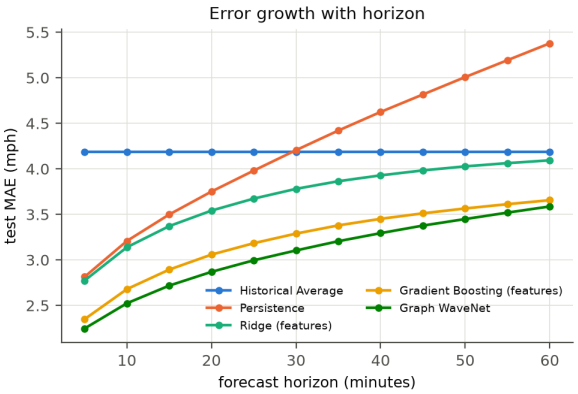
5. What the project found

Test-set MAE in mph (lower is better) on 6,850 windows × 207 sensors. Bottom rows: published results under the same protocol.

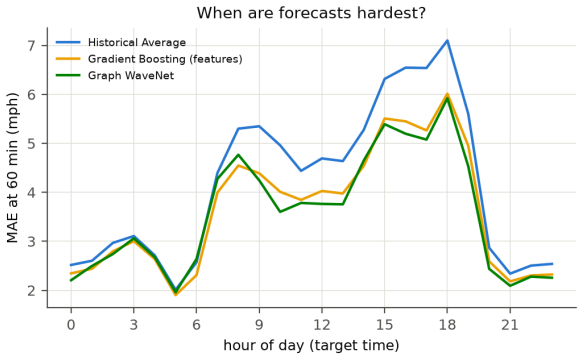
Model	15 min	30 min	60 min	Average	95% CI (avg)
Historical Average	4.19	4.19	4.19	4.19	3.89 – 4.41
Persistence	3.50	4.21	5.38	4.24	3.96 – 4.43
Ridge (features)	3.37	3.78	4.09	3.69	3.47 – 3.83
Gradient Boosting (features)	2.89	3.29	3.66	3.22	3.01 – 3.36
Graph WaveNet	**2.72**	**3.10**	**3.59**	**3.07**	**2.89 – 3.20**
DCRNN (Li et al., 2018)	2.77	3.15	3.60	–	
Graph WaveNet (Wu et al., 2019)	2.69	3.07	3.53	–	

The main findings, in one paragraph each

- **Graph WaveNet is best, and significantly so.** Its average error is 3.07 mph, 4.5% below gradient boosting. Diebold–Mariano tests reject equal accuracy at all three horizons ($p < 0.001$), and the result is within 0.03–0.06 mph of the published Graph WaveNet numbers despite a shorter training run.
- **Good features get a classical model most of the way.** Gradient boosting with the 34 engineered features (3.22 mph) beats the published deep FC-LSTM at every horizon and is close behind Graph WaveNet at 60 minutes (3.66 vs 3.59 mph). Our Graph WaveNet also beats the published DCRNN at all three horizons.
- **"The usual speed for this time of week" is the single most useful feature** (permutation importance), especially 60 minutes ahead, where the current state matters less.
- **Missing data hurts the deep model more.** With the latest reading missing, Graph WaveNet's 60-minute error is 5.36 mph against gradient boosting's 4.50.



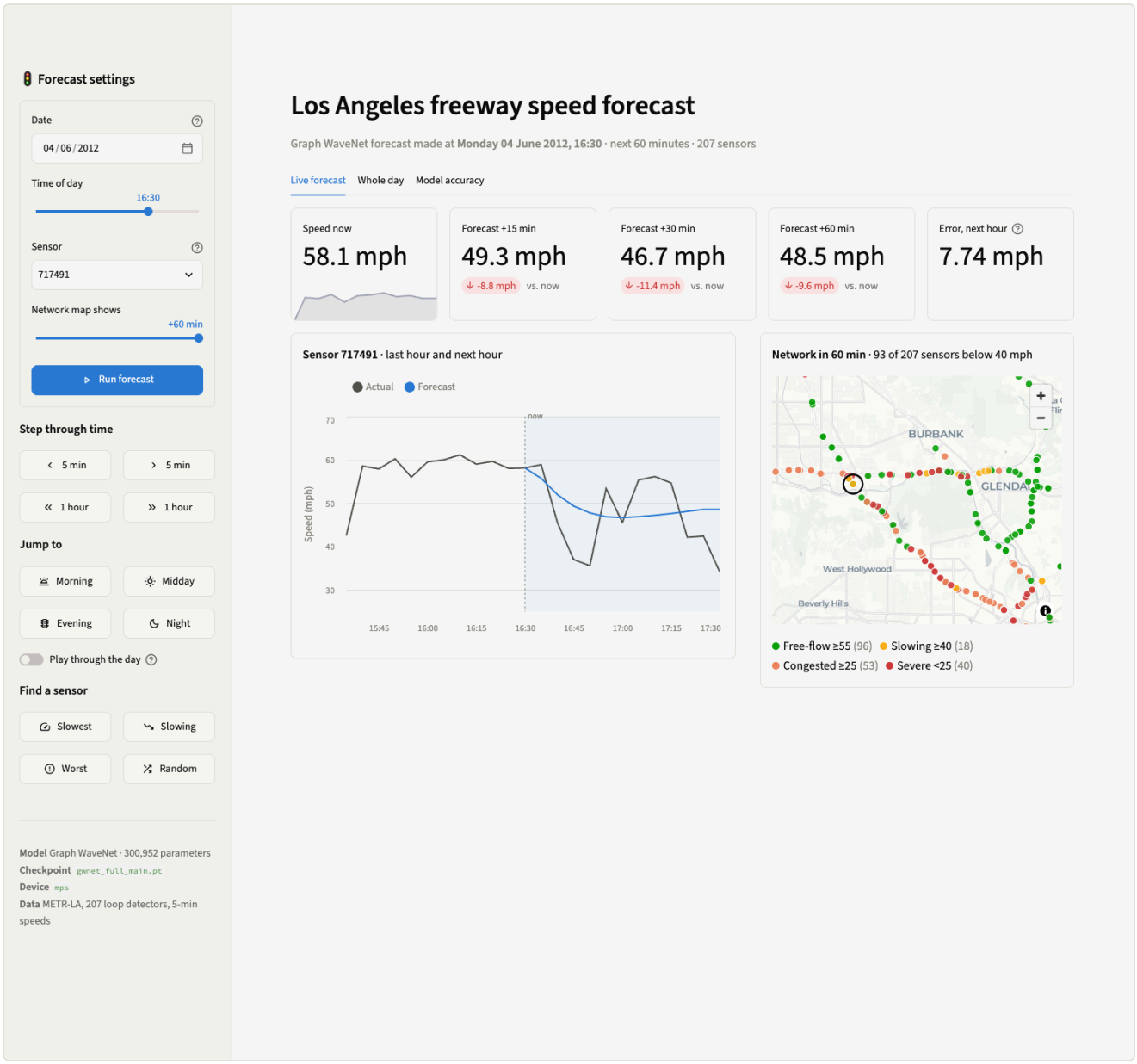
Error grows with horizon. Persistence is good 5 minutes ahead but worst at 60; the Historical Average is flat. The learned models are lowest at every step.



When forecasts are hardest. 60-minute errors peak in the evening rush (15:00–19:00) and are lowest at night.

6. The interactive web app

`app/app.py` turns the trained Graph WaveNet into a page you can explore at `http://localhost:8501`. It is built with **Streamlit** (page and controls), **Altair** (interactive charts) and **pydeck** (the map).

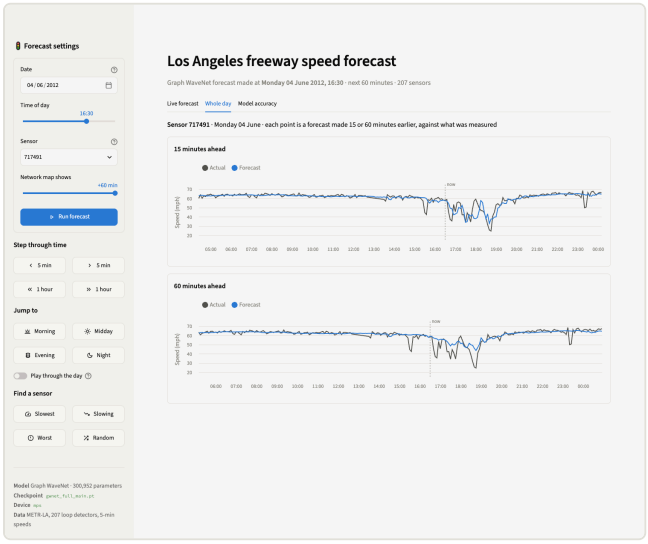


Live forecast tab. Left: forecast settings and quick actions. Top: current speed and the forecast 15, 30 and 60 minutes ahead at the chosen sensor. Bottom left: last hour (grey) and next hour (actual grey, forecast blue). Bottom right: every sensor on a street map, coloured by forecast congestion.

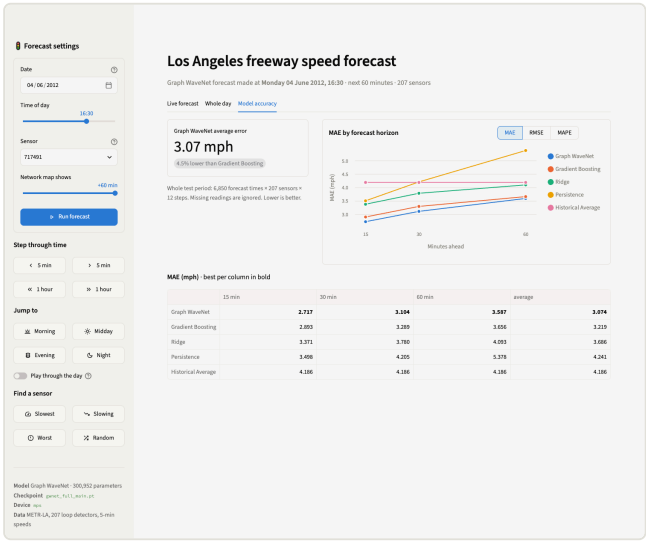
How it works

1. **On start-up** it loads the dataset and the Graph WaveNet checkpoint once and keeps them in memory (`st.cache_resource`), picking the GPU when one is available.
2. **When you click Run forecast** it finds the test window for the chosen date and time, runs the model on that one window (about 6 ms on the M3 GPU), and draws the result. Nothing recalculates until you click, so you can change several settings first.
3. **Quick-action buttons** apply immediately: step ±5 minutes or ±1 hour, jump to the morning rush, midday, evening rush or night, or let *Play through the day* advance the clock every second.
4. **Find a sensor** buttons pick the slowest sensor now, the one forecast to slow down most, the one with the worst forecast this hour, or a random one. You can also click any dot on the map.

5. **Whole day** shows the forecast made 15 and 60 minutes earlier against what happened, for every 5-minute step of the day.
6. **Model accuracy** shows the saved test-set scores of Graph WaveNet and the baselines (from `results/metrics/`), by MAE, RMSE or MAPE.



Whole day tab. Each point is a forecast made earlier, plotted against the measured speed.



Model accuracy tab. Graph WaveNet against the four baselines, with a switch between error metrics.

7. Repository map

Path	Purpose
`src/traffic/data.py`	Load speeds, sensor list, distances; Gaussian graph; transition matrices; chronological split; scaler
`src/traffic/features.py`	Forward-fill, weekly profile, the 34 causal features
`src/traffic/metrics.py`	Masked MAE/RMSE/MAPE, Diebold–Mariano test
`src/traffic/models/`	`baselines.py` (HA, Persistence), `classical.py` (one regressor per horizon), `lstm.py`, `gwnet.py`
`src/traffic/trainer.py`	Window sampler on the GPU, masked-MAE training loop, early stopping, checkpoints
`src/traffic/eda.py`, `plotting.py`	EDA statistics and figures; shared plot style
`scripts/`	One entry point per experiment (see Section 3)
`tests/`	23 pytest tests: no future leakage, split order, scaler fitted on train only, graph properties, masking, model shapes
`results/`	`metrics/` (JSON per model), `tables/`, `figures/`, `history/`, `logs/`, `predictions/` (not in git)
`app/app.py`, `.streamlit/config.toml`	The web app and its theme
`report/`	IEEE-format LaTeX paper
`docs/`	Literature review, viva preparation, these PDFs (sources in `docs/pdf_src/`)
`notebooks/01_eda.ipynb`	Executed, narrated exploratory analysis

Tools and libraries

Tool	Used for
Python 3.14	Everything
NumPy, pandas	Arrays, time indexes, windows, tables
h5py	Reading the legacy HDF5 speed file
SciPy, statsmodels	Normal distribution for the DM test; ADF stationarity test and autocorrelation
scikit-learn	Ridge, HistGradientBoostingRegressor, StandardScaler, permutation importance
PyTorch (MPS backend)	LSTM and Graph WaveNet, trained on the Apple M3 GPU
Matplotlib	All report figures
Streamlit, Altair, pydeck	Web app, interactive charts, map
pytest	Unit tests
LaTeX (Tectonic)	IEEE paper

Glossary

Loop detector	A wire loop in the road surface that senses passing cars; here it reports 5-minute average speed.
Horizon	How far ahead the forecast is (5 to 60 minutes; step 1 to 12).
Anchor	The "now" time of a window: the last input step.
Masking	Leaving missing (0) values out of the loss and the scores.
Leakage	Accidentally letting information from the future (or the test set) into training. It makes scores look better than they really are.
Ablation	Removing or adding one component at a time to measure what it contributes.
Checkpoint	The saved model weights from the epoch with the best validation score.
MAE / RMSE / MAPE	Mean absolute error (mph), root mean squared error (mph), mean absolute percentage error (%).